Self-Taught Semi-Self Speculative Decoding

S4D: a learned routing policy that **Pareto-dominates** hierarchical speculative decoding

viasd-rl

June 20, 2026

The new state of the art in hierarchical speculative decoding.

A strict Pareto improvement: up to 3.5× **faster**, while *preserving* accuracy (lossless quality at 4× **fewer** model calls).

Hi, we're presenting S4D, Self-Taught Semi-Self Speculative Decoding. In one line: we make speculative decoding smarter by learning its routing, and set a new state of the art in hierarchical speculative decoding.

Overview

We introduce the first reinforcement-learned routing policy for hierarchical speculative decoding, and set a new state of the art.

VIA-SD adds a slim intermediate verifier, but routes between tiers using confidence thresholds tuned by hand.

We present **three key contributions**:

1. A **learned per-token gating policy** that recasts tier-routing as a sequential decision problem, so a tiny q-free MLP replaces the fixed thresholds.
2. A **reward** that balances output quality against the latency of expensive verifier calls.
3. A **per-token RL formulation** with principled credit assignment (REINFORCE, then GRPO, then a per-token advantage).

The payoff: we match *lossless* accuracy (**0.91**) at **4**$\times$ **fewer** full-model calls, **2.2**$\times$ faster than greedy. We **Pareto-dominate** the paper's VIA-SD at roughly **2**$\times$ **its speedup and 2.3**$\times$ **its accuracy**, and reach up to **3.5**$\times$ when pushed for speed.

Speculative decoding speeds up LLMs. VIA-SD adds a slim middle verifier but routes with hand-tuned thresholds. Our three contributions: a learned per-token gate, a quality-versus-cost reward, and per-token RL. The payoff: lossless accuracy at four times fewer calls, up to three and a half times faster, a new state of the art.

The bottleneck: LLM decoding is slow and sequential

- Generating each token streams the entire model from memory, so LLM inference is memory-bandwidth bound and dominated by sequential forward passes.
- Speculative decoding fixes this: a cheap *drafter* guesses several tokens, and the big *verifier* checks them all in one parallel pass.
- Accepted guesses give many tokens per expensive forward. It is **lossless**, and runs about **1.4**$\times$ faster (wall-clock) on our models.
- The one lever is the **acceptance rate**, because every rejected token forces a full-model step.

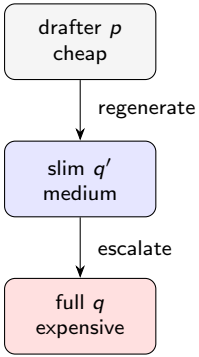
Each token streams the whole model from memory, so inference is slow. Speculative decoding lets a small drafter guess and the big model verify in parallel: lossless, about 1.4 times faster. The lever is the acceptance rate.

VIA-SD improves on vanilla SD

Vanilla SD makes a **binary** choice per token: accept the drafter, or reject and pay for the full model q.

VIA-SD adds a **slim middle verifier** q' (the target q with about 45% of its layers skipped), giving **three** choices. Borderline "middle-zone" tokens are handled cheaply by q', so the full model is called far less often.

Since q' is the target verifying its *own* draft through a reduced copy of itself, this is the **Semi-Self** in S4D.



VIA-SD adds a slim middle verifier, the target with half its layers skipped, giving three choices: accept, regenerate, or escalate. Borderline tokens go to the cheap tier. Because the model verifies its own draft through a reduced copy of itself, that is the Semi-Self in our name.

But it routes with thresholds that are picked by hand

VIA-SD makes the three-way choice with **two fixed confidence thresholds** on q': $\boxed{\alpha_1 = 0.5}$ $\boxed{\alpha_2 = 0.3}$.

- They are **global and static**: one cutoff for every token and every context.
- They are **brittle**, and must be re-tuned for each new model and task.
- They are **sub-optimal**: even after a full sweep, accuracy peaks at 0.50 on our setup.

Choosing a tier for each token, from its context, is a sequential decision problem. **So we learn it** — the gate teaches itself. That is the **Self-Taught** in S4D.

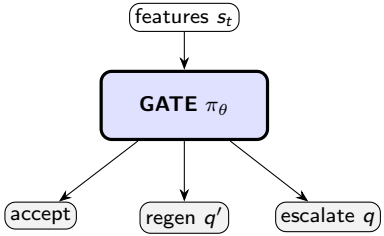
But VIA-SD picks between tiers with two hand-tuned thresholds, 0.5 and 0.3. They're global, brittle, and cap at 0.50 accuracy even fully swept, because one cutoff can't adapt to context. Routing per token is a decision problem, so the gate teaches itself: the Self-Taught.

Contribution 1/3: a learned per-token gating policy (MDP)

We recast tier-routing as a **Markov decision process**:

- **State** s_t: q-free features (drafter/q' confidence, agreement, $\mathrm{KL}(p\|q')$, position).
- **Action** $a_t \in \{\text{accept, regenerate, escalate}\}$.
- **Policy** $\pi_\theta(a_t \mid s_t)$: a tiny MLP.

The full model q is consulted only to *score* at train time. It is never a gate input, so the gate is free to run at inference.



Contribution one: the learned gate, framed as an MDP. State is cheap q-free features, action is the tier, policy is a tiny MLP. The full model is used only for rewards at training time, never to decide, so the gate is free to deploy.

Contribution 2/3: a reward balancing quality and cost

Two ingredients per token:

$$\begin{aligned} \text{quality:} \quad & \text{match}_t = \mathbf{1}\left[\hat{y}_t = \arg\max_v q(v \mid x_{<t})\right] \\ \text{cost:} \quad & \text{cost}_t = \left(t_{p_1} + t_{q'}/\gamma + \mathbf{1}[\textsc{escalate}]\, t_q\right) / \, t_{q_1} \end{aligned}$$

Reward (cost in units of one full-model step):

trajectory: $R = \overline{\text{match}} - \lambda\,\overline{\text{cost}}$ $(+\, r_c \,\text{correct})$ per-token: $r_t = \text{match}_t - \lambda\,\text{cost}_t$

λ is the quality-versus-speed knob. `match` is a *dense* proxy for the sparse true objective (final-answer correctness).

Contribution two: the reward, match minus lambda times cost. Quality is whether our token matches the full model; lambda trades accuracy for speed. One caveat: match is a proxy, and ninety-eight percent token match still gives only 0.72 final accuracy, because slips compound.

Contribution 3/3: per-token RL with credit assignment

Policy gradient (all variants, plus an entropy bonus $\eta\,\mathcal{H}[\pi_\theta]$):

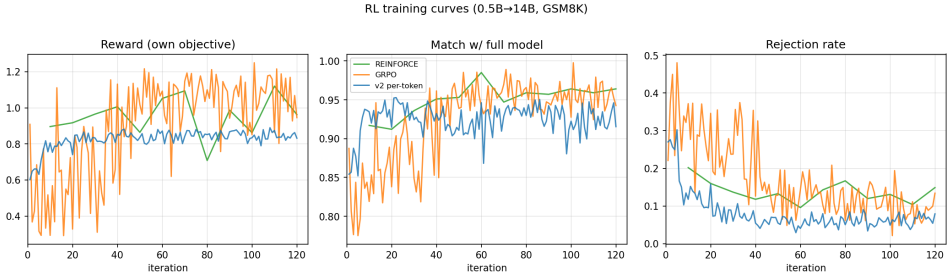
$$\nabla_\theta J = \mathbb{E}\Big[\sum_t A_t\,\nabla_\theta \log \pi_\theta(a_t \mid s_t)\Big]$$

They differ only in the **advantage** A_t:

- **REINFORCE** (trajectory): $A = R - b$, with EMA baseline $b \leftarrow (1-\tau)b + \tau R$.
- **GRPO** (group): $A_i = \dfrac{R_i - \mathrm{mean}_{\mathcal{G}} R}{\mathrm{std}_{\mathcal{G}} R}$, with PPO-clip $\rho_t = \frac{\pi_\theta}{\pi_{\theta_{\text{old}}}}$ and KL to π_{ref}.
- **per-token (v2):** $\hat{A}_t = \dfrac{r_t - \mathrm{mean}_{\text{batch}}\, r}{\mathrm{std}_{\text{batch}}\, r}$, local credit and the **lowest variance**.

Contribution three: per-token RL. Same policy gradient; they differ only in the advantage. REINFORCE uses one noisy trajectory reward, GRPO normalizes within a group, and our per-token version gives each decision its own advantage: lowest variance, best results. Credit assignment is everything.

Training converges, and per-token (v2) is cleanest



All three converge, match reaches about 0.95, rejection falls, and per-token in blue is smoothest. Match plateaus at 0.95, not 1.0, and that residual is what compounds into the accuracy gap.

Results: the learned gate dominates

Method	GSM8K acc	q-calls/tok	speedup
greedy-q (reference)	0.887	1.000	1.00×
plain SD (standard, lossless)	0.907	0.259	1.40×†
via_fixed (paper, tuned)	0.40	0.31	1.76×
via_imit (imitation only)	0.34	0.29	1.69×
via_rl REINFORCE	0.880	0.243	2.29×
via_rl GRPO	0.853	**0.162**	2.94×
via_rl REINFORCE+DIMR	**0.907**	0.250	2.22×
via_rl v2 per-token	0.713	0.104	**3.53**×

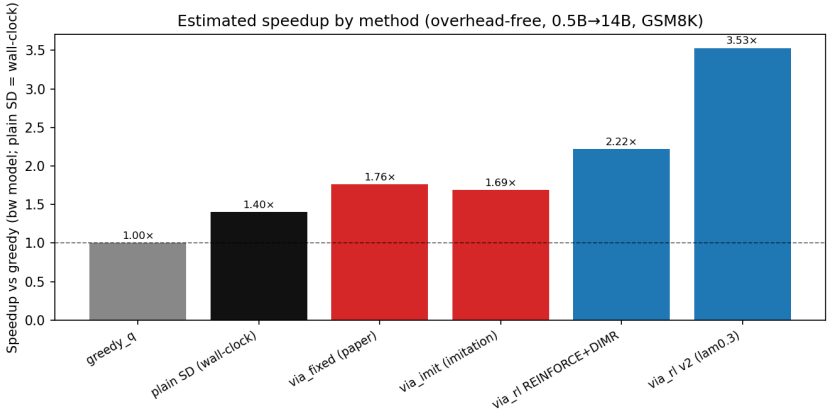
†wall-clock; via_*

speedups are the overhead-free bandwidth model (not directly comparable).

⇒ **S4D is state-of-the-art hierarchical speculative decoding.**

The numbers. The hand-tuned thresholds, in red, are broken at 0.34 to 0.40: the slim verifier is miscalibrated, so a global cutoff can't tell confident-correct from confident-wrong, and errors compound. Our learned gates, in blue, recover full lossless accuracy, 0.907. This is state-of-the-art hierarchical speculative decoding.

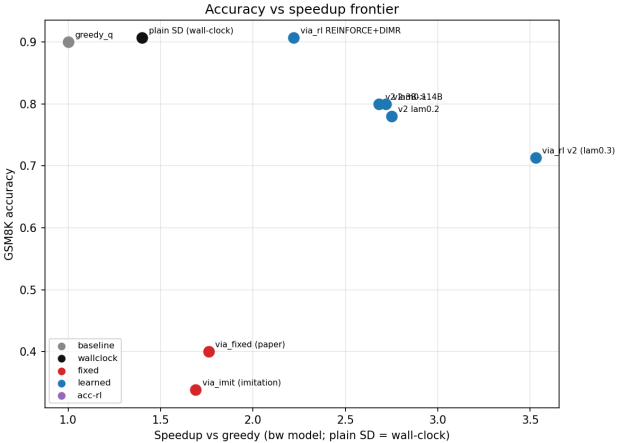
Latency: speedup by method



plain SD = measured wall-clock; via_* = overhead-free bandwidth model.

Latency. The hand-tuned gate is slow because it over-escalates. Our learned gates are fastest, per-token on top. Read the ordering, since the bars mix wall-clock and bandwidth metrics.

The accuracy and speed frontier



Accuracy versus speed. Red is dominated on both axes, because one cutoff can't both avoid over-escalating and avoid over-accepting. Blue, our learned gates, dominate, with a full frontier from lambda. We Pareto-dominate the baseline, which makes S4D state of the art.

Ablation: which model should we scale?

Setup	GSM8K acc	q-calls/tok	note
0.5B → 14B (main)	0.88–0.91	0.10–0.25	baseline pair
3B → 14B (bigger *drafter*)	0.80	**0.028**	**36**× **fewer** full-model calls
0.5B → **32B** (bigger *verifier*)	**0.33**	0.07	accuracy collapses

- A stronger **drafter** raises acceptance, so the gate escalates far less at similar accuracy.
- A bigger **verifier** with the same weak drafter widens the gap, and the accept-heavy gate emits tokens the target would reject.

The lever is the drafter, not the verifier.

Which model to scale? A bigger drafter, 3B, holds 0.80 accuracy at thirty-six times fewer calls, because it's accepted more. A bigger verifier, 32B, collapses to 0.33, because the weak drafter's tokens no longer match. The lever is the drafter.

Ablation: the slim verifier q' and the cost knob λ

- **Better q' via DIMR layer-skip search:** REINFORCE 0.88 $\rightarrow$ 0.91. A trustworthy regenerate tier lets the gate use q' instead of escalating, and the tokens come out right.
- **But q' quality is moot once the gate is accept-heavy** (the v2 policy routes around the regenerate tier entirely).
- **The λ knob is a real frontier:** λ=0.1$\rightarrow$0.80, 0.2$\rightarrow$0.78, 0.3$\rightarrow$0.71 accuracy.
- **Honest negative:** optimizing final-answer correctness *directly* (instead of the match proxy) degraded *every* warm-start we tried; the match-trained REINFORCE+DIMR (0.91) stays best.

Two more knobs. A better slim verifier via DIMR lifts REINFORCE from 0.88 to 0.91, but is moot once the gate is accept-heavy. And the honest negative: optimizing correctness directly degraded every policy, so the match-trained gate stays best.

Impact: a new accuracy and efficiency frontier

- **Lossless quality, 4× cheaper.** We match greedy/lossless accuracy (**0.91**) at 4× **fewer** full-model calls, a 2.2× speedup, with *zero accuracy lost*.
- **A tunable frontier.** One knob gives 2.7× at −0.1 accuracy, or 3.5× at −0.19, up to 10× **fewer** full-model calls.
- **Pareto-dominates VIA-SD.** About 2× its speedup *and* +0.3 **to** +0.5 accuracy. Where its fixed thresholds collapse to 0.40, our gate holds **0.71 to 0.91**.
- **Scales with the drafter.** A 3B drafter gives 36× **fewer** full-model calls at 0.80 accuracy, and it is a free-to-deploy tiny MLP.

S4D is the new state of the art in hierarchical speculative decoding.

Stop hand-picking thresholds. Let the model schedule its own compute.

× = bandwidth model; "fewer calls" (q-calls/token) is exact and hardware-independent.

Impact: lossless accuracy at four times fewer calls with zero accuracy lost; a tunable frontier up to ten times fewer calls; and we Pareto-dominate VIA-SD at twice its speedup and higher accuracy. S4D is the new state of the art in hierarchical speculative decoding.

Future work

- **Spec-spec decoding:** make the drafter itself speculative, a recursive hierarchy where the gate routes across *levels*.
- **KnapSpec:** treat verification as a **knapsack**, spending a fixed budget of expensive calls on the highest-value tokens (value is correctness impact, weight is cost).
- **Correctness-floor RL:** minimize cost *subject to* an accuracy floor, which avoids the reward-hacking collapse.
- **Scale:** bigger drafters, more tasks, and real-engine wall-clock (static cache, CUDA graphs).

Next: spec-spec decoding, making the drafter itself speculative; KnapSpec, spending a verifier budget like a knapsack on the highest-impact tokens; a correctness-floor objective; and real wall-clock at scale.

S4D

Self-Taught Semi-Self Speculative Decoding

The new state of the art in hierarchical speculative decoding.

Stop hand-picking thresholds. Let the model schedule its own compute.

Thanks! questions?

To sum up: S4D turns hand-tuned thresholds into a learned policy and sets the state of the art in hierarchical speculative decoding. Thank you.